

SSN College of Engineering

LABORATORY RECORD

Subject: Web Application Development Laboratory

Exercise 5: Course Registration Web Application using Spring Boot, MongoDB and Postman

SUBMITTED BY:

Name: Vijayaraaghavan K S
Register Number: 3122247001066
Branch: Department of CSE
Semester / Year: V Semester / III Year

Subject Code: ICS1511

Batch: 2024–2029

1. Problem Description

Scenario

The college wants a web application that allows students to view available courses, register for courses, remove registered courses, and lets the administration run analytical queries across multiple MongoDB collections to understand registration patterns.

The application is built using **Spring Boot** (REST API layer), **MongoDB** (document database, database name **CourseDB**) and is tested using **Postman**.

MongoDB Collections

- **Students** – `studentId`, `studentName`, `department`, `year`
- **Courses** – `courseId`, `courseName`, `credits`, `department`, `instructorId`
- **Registrations** – `registrationId`, `studentId`, `courseId`, `semester`, `academicYear`
- **Instructors** – `instructorId`, `instructorName`, `experience`, `department`

Required REST APIs

Operation	Method	URL
Retrieve All Courses	GET	<code>/api/courses</code>
Register for a Course	POST	<code>/api/register</code>
Get Registered Courses for Student	GET	<code>/api/students/{studentId}/courses</code>
Remove Registered Course	DELETE	<code>/api/students/{studentId}/courses/{courseId}</code>
Most Popular Courses	GET	<code>/api/analytics/popular-courses</code>
Department Wise Course Registrations	GET	<code>/api/analytics/department-registrations</code>
Students Registered for More Than 2 Courses	GET	<code>/api/analytics/multiple-courses</code>
Total Credits Registered Per Student	GET	<code>/api/analytics/student-credits</code>

Technology Stack

- Spring Boot 3.5.4 (Java 17), `spring-boot-starter-web`, `spring-boot-starter-data-mongodb`, `spring-boot-starter-validation`
- MongoDB (local instance, database `CourseDB`, verified with MongoDB Compass)
- Lombok for model boilerplate
- Postman for REST API testing
- Maven for build/dependency management

2. Source Code

2.1 application.properties

```
spring.data.mongodb.uri=mongodb://localhost:27017/CourseDB
server.port=8080
```

2.2 Model Classes

File: Student.java

```
1 package com.ssn.courseregistration.model;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6 import org.springframework.data.annotation.Id;
7 import org.springframework.data.mongodb.core.mapping.Document;
8
9 @Data
10 @NoArgsConstructor
11 @AllArgsConstructor
12 @Document(collection = "students")
13 public class Student {
14
15     @Id
16     private String studentId;
17
18     private String studentName;
19     private String department;
20     private int year;
21 }
```

File: Course.java

```
1 package com.ssn.courseregistration.model;
2
3 import lombok.*;
4 import org.springframework.data.annotation.Id;
5 import org.springframework.data.mongodb.core.mapping.Document;
6
7 @Data
8 @NoArgsConstructor
9 @AllArgsConstructor
10 @Document(collection = "courses")
11 public class Course {
12
13     @Id
14     private String courseId;
15
16     private String courseName;
17     private int credits;
18     private String department;
19     private String instructorId;
20 }
```

File: Registration.java

```
1 package com.ssn.courseregistration.model;
2
3 import lombok.*;
4 import org.springframework.data.annotation.Id;
5 import org.springframework.data.mongodb.core.mapping.Document;
6
7 @Data
8 @NoArgsConstructor
9 @AllArgsConstructor
10 @Document(collection = "registrations")
11 public class Registration {
12
13     @Id
14     private String registrationId;
15
16     private String studentId;
17     private String courseId;
18     private String semester;
19     private String academicYear;
20 }
```

File: Instructor.java

```
1 package com.ssn.courseregistration.model;
2
3 import lombok.*;
4 import org.springframework.data.annotation.Id;
5 import org.springframework.data.mongodb.core.mapping.Document;
6
7 @Data
8 @NoArgsConstructor
9 @AllArgsConstructor
10 @Document(collection = "instructors")
11 public class Instructor {
12
13     @Id
14     private String instructorId;
15
16     private String instructorName;
17     private int experience;
18     private String department;
19 }
```

File: RegistrationRequest.java (DTO)

```
1 package com.ssn.courseregistration.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import lombok.Data;
5
6 @Data
7 public class RegistrationRequest {
8
9     @NotBlank(message = "Student ID cannot be empty")
10     private String studentId;
11
12     @NotBlank(message = "Course ID cannot be empty")
13     private String courseId;
```

```
14 }
```

2.3 Repository Interfaces

File: RegistrationRepository.java

```
1 package com.ssn.courseregistration.repository;
2
3 import com.ssn.courseregistration.model.Registration;
4 import org.springframework.data.mongodb.repository.MongoRepository;
5
6 import java.util.List;
7
8 public interface RegistrationRepository
9     extends MongoRepository<Registration,String> {
10
11     List<Registration> findByStudentId(String studentId);
12
13     void deleteByStudentIdAndCourseId(
14         String studentId,
15         String courseId
16     );
17 }
```

The StudentRepository, CourseRepository and InstructorRepository interfaces simply extend MongoRepository<T, String> to provide standard CRUD operations for their respective collections.

2.4 Service Classes

File: RegistrationService.java

```
1 package com.ssn.courseregistration.service;
2
3 import com.ssn.courseregistration.model.Registration;
4 import com.ssn.courseregistration.repository.RegistrationRepository;
5 import org.springframework.stereotype.Service;
6 import org.springframework.data.mongodb.core.MongoTemplate;
7 import jakarta.validation.Valid;
8 import jakarta.validation.constraints.NotBlank;
9
10 import java.time.LocalDate;
11 import java.util.List;
12
13 @Service
14 public class RegistrationService {
15
16     private final MongoTemplate mongoTemplate;
17     private final RegistrationRepository registrationRepository;
18
19     public RegistrationService(
20         RegistrationRepository registrationRepository,
21         MongoTemplate mongoTemplate) {
22
23         this.registrationRepository = registrationRepository;
24         this.mongoTemplate = mongoTemplate;
25     }
26
27     public Registration register(
28         @NotBlank(message = "Student ID cannot be empty")
```

```

29         String studentId,
30         @NotBlank(message = "Course ID cannot be empty")
31         String courseId) {
32
33         Registration registration =
34             new Registration();
35
36         registration.setStudentId(studentId);
37         registration.setCourseId(courseId);
38
39         LocalDate today = LocalDate.now();
40         int year = today.getYear();
41         if (today.getMonthValue() >= 7) {
42             registration.setSemester("Odd");
43             registration.setAcademicYear(year + "-" + (year + 1));
44         } else {
45             registration.setSemester("Even");
46             registration.setAcademicYear((year - 1) + "-" + year);
47         }
48
49         return registrationRepository.save(registration);
50     }
51
52     public List<Registration> getCourses(
53         String studentId) {
54
55         return registrationRepository
56             .findByStudentId(studentId);
57     }
58
59     public void removeCourse(
60         String studentId,
61         String courseId) {
62
63         registrationRepository
64             .deleteByStudentIdAndCourseId(
65                 studentId,
66                 courseId
67             );
68     }
69
70     public List<Registration> getAll() {
71         return registrationRepository.findAll();
72     }
73
74     public String getDatabaseName() {
75         return mongoTemplate.getDb().getName();
76     }
77 }

```

CourseService and StudentService follow the same pattern – thin wrappers around their respective repositories exposing save, getAll, getById and delete.

File: AnalyticsService.java

This class implements the four complex, multi-collection MongoDB aggregation-pipeline queries (items e–h of the specification) using Spring Data MongoDB's `MongoTemplate` and the Aggregation framework (`$group`, `$lookup`, `$unwind`, `$project`, `$match`, `$sort`).

```

1 package com.ssn.courseregistration.service;
2
3 import org.bson.Document;
4 import org.springframework.data.domain.Sort;

```

```
5 import org.springframework.data.mongodb.core.MongoTemplate;
6 import org.springframework.data.mongodb.core.aggregation.*;
7 import org.springframework.data.mongodb.core.query.Criteria;
8 import org.springframework.stereotype.Service;
9
10 import java.util.List;
11
12 @Service
13 public class AnalyticsService {
14
15     private final MongoTemplate mongoTemplate;
16
17     public AnalyticsService(
18         MongoTemplate mongoTemplate) {
19
20         this.mongoTemplate = mongoTemplate;
21     }
22
23     // e. Most Popular Courses
24     public List<Document> getPopularCourses() {
25
26         Aggregation aggregation =
27             Aggregation.newAggregation(
28
29                 Aggregation.group("courseId")
30                     .count().as("totalRegistrations"),
31
32                 Aggregation.lookup(
33                     "courses", "_id", "_id", "course"
34                 ),
35
36                 Aggregation.unwind("course"),
37
38                 Aggregation.project()
39                     .and("course.courseName").as("courseName")
40                     .and("totalRegistrations").as("totalRegistrations")
41                     .andExclude("_id"),
42
43                 Aggregation.sort(
44                     Sort.Direction.DESC, "totalRegistrations"
45                 )
46             );
47
48         return mongoTemplate.aggregate(
49             aggregation, "registrations", Document.class
50         ).getMappedResults();
51     }
52
53     // f. Department Wise Course Registrations
54     public List<Document> getDepartmentWiseRegistrations() {
55
56         Aggregation aggregation =
57             Aggregation.newAggregation(
58
59                 Aggregation.lookup(
60                     "students", "studentId", "_id", "student"
61                 ),
62
63                 Aggregation.unwind("student"),
64
65                 Aggregation.group("student.department")
66                     .addToSet("studentId").as("students")
67             );
68     }
```

```

67         .count().as("totalCoursesTaken"),
68
69         Aggregation.project()
70         .and("_id").as("department")
71         .and("totalCoursesTaken").as("totalCoursesTaken")
72         .and(ArrayOperators.arrayOf("students").length())
73         .as("totalStudents")
74         .andExclude("_id")
75     );
76
77     return mongoTemplate.aggregate(
78         aggregation, "registrations", Document.class
79     ).getMappedResults();
80 }
81
82 // g. Students Registered for More Than 2 Courses
83 public List<Document> getStudentsWithMultipleCourses() {
84
85     Aggregation aggregation =
86         Aggregation.newAggregation(
87
88         Aggregation.group("studentId")
89             .count().as("courseCount"),
90
91         Aggregation.match(
92             Criteria.where("courseCount").gt(2)
93         ),
94
95         Aggregation.lookup(
96             "students", "_id", "_id", "student"
97         ),
98
99         Aggregation.unwind("student"),
100
101         Aggregation.project()
102             .and("student.studentName").as("studentName")
103             .and("courseCount").as("courseCount")
104             .andExclude("_id"),
105
106         Aggregation.sort(
107             Sort.Direction.DESC, "courseCount"
108         )
109     );
110
111     return mongoTemplate.aggregate(
112         aggregation, "registrations", Document.class
113     ).getMappedResults();
114 }
115
116 // h. Total Credits Registered Per Student
117 public List<Document> getTotalCreditsPerStudent() {
118
119     Aggregation aggregation =
120         Aggregation.newAggregation(
121
122         Aggregation.lookup(
123             "courses", "courseId", "_id", "course"
124         ),
125
126         Aggregation.unwind("course"),
127
128         Aggregation.group("studentId")
129             .sum("course.credits").as("totalCredits"),

```

```

130
131         Aggregation.lookup(
132             "students", "_id", "_id", "student"
133         ),
134
135         Aggregation.unwind("student"),
136
137         Aggregation.project(
138             .and("student.studentName").as("studentName")
139             .and("totalCredits").as("totalCredits")
140             .andExclude("_id"),
141
142         Aggregation.sort(
143             Sort.Direction.DESC, "totalCredits"
144         )
145     );
146
147     return mongoTemplate.aggregate(
148         aggregation, "registrations", Document.class
149     ).getMappedResults();
150 }
151 }

```

2.5 Controller Classes

File: CourseController.java

```

1 package com.ssn.courseregistration.controller;
2
3 import com.ssn.courseregistration.model.Course;
4 import com.ssn.courseregistration.service.CourseService;
5 import org.springframework.web.bind.annotation.*;
6
7 import java.util.List;
8
9 @RestController
10 @RequestMapping("/api/courses")
11 public class CourseController {
12
13     private final CourseService service;
14
15     public CourseController(CourseService service) {
16         this.service = service;
17     }
18
19     @PostMapping
20     public Course createCourse(@RequestBody Course course) {
21         return service.save(course);
22     }
23
24     @GetMapping
25     public List<Course> getAllCourses() {
26         return service.getAll();
27     }
28
29     @GetMapping("/{id}")
30     public Course getCourse(@PathVariable String id) {
31         return service.getId(id);
32     }
33
34     @DeleteMapping("/{id}")
35     public String deleteCourse(@PathVariable String id) {

```

```
36     service.delete(id);
37     return "Course Deleted";
38 }
39 }
```

File: RegistrationController.java

```
1  package com.ssn.courseregistration.controller;
2
3  import com.ssn.courseregistration.dto.RegistrationRequest;
4  import com.ssn.courseregistration.model.Registration;
5  import com.ssn.courseregistration.service.RegistrationService;
6  import org.springframework.web.bind.annotation.*;
7  import jakarta.validation.Valid;
8
9  import java.util.List;
10
11 @RestController
12 @RequestMapping("/api")
13 public class RegistrationController {
14
15     private final RegistrationService service;
16
17     public RegistrationController(RegistrationService service) {
18         this.service = service;
19     }
20
21     @PostMapping("/register")
22     public Registration registerCourse(
23         @Valid @RequestBody RegistrationRequest request) {
24
25         return service.register(
26             request.getStudentId(),
27             request.getCourseId()
28         );
29     }
30
31     @GetMapping("/students/{studentId}/courses")
32     public List<Registration> getCourses(
33         @PathVariable String studentId) {
34
35         return service.getCourses(studentId);
36     }
37
38     @DeleteMapping("/students/{studentId}/courses/{courseId}")
39     public String removeCourse(
40         @PathVariable String studentId,
41         @PathVariable String courseId) {
42
43         service.removeCourse(studentId, courseId);
44         return "Course Removed Successfully";
45     }
46 }
```

File: AnalyticsController.java

```
1  package com.ssn.courseregistration.controller;
2
3  import com.ssn.courseregistration.service.AnalyticsService;
4  import org.bson.Document;
```

```

5 import org.springframework.web.bind.annotation.*;
6
7 import java.util.List;
8
9 @RestController
10 @RequestMapping("/api/analytics")
11 public class AnalyticsController {
12
13     private final AnalyticsService service;
14
15     public AnalyticsController(AnalyticsService service) {
16         this.service = service;
17     }
18
19     @GetMapping("/popular-courses")
20     public List<Document> getPopularCourses() {
21         return service.getPopularCourses();
22     }
23
24     @GetMapping("/department-registrations")
25     public List<Document> getDepartmentRegistrations() {
26         return service.getDepartmentWiseRegistrations();
27     }
28
29     @GetMapping("/multiple-courses")
30     public List<Document> getStudentsWithMultipleCourses() {
31         return service.getStudentsWithMultipleCourses();
32     }
33
34     @GetMapping("/student-credits")
35     public List<Document> getTotalCreditsPerStudent() {
36         return service.getTotalCreditsPerStudent();
37     }
38 }

```

StudentController follows the identical CRUD pattern as CourseController, exposing /api/students.

3. Sample Data Seeded in MongoDB (CourseDB)

The following documents were inserted into the demo/CourseDB database (visible in MongoDB Compass) before testing the APIs from Postman.

```

1 // students
2 { _id: "S101", studentName: "Arjun Kumar", department: "CSE", year: 3 }
3 { _id: "S102", studentName: "Priya Sharma", department: "IT", year: 3 }
4 { _id: "S103", studentName: "Rahul Verma", department: "ECE", year: 2 }
5 { _id: "S104", studentName: "Sneha Patil", department: "CSE", year: 3 }
6 { _id: "S105", studentName: "Karthik Raja", department: "IT", year: 2 }
7
8 // courses
9 { _id: "C101", courseName: "Data Structures", credits: 4, department: "CSE",
10   instructorId: "I101" }
11 { _id: "C102", courseName: "Database Systems", credits: 3, department: "CSE",
12   instructorId: "I104" }
13 { _id: "C103", courseName: "Computer Networks", credits: 4, department: "ECE",
14   instructorId: "I103" }
15 { _id: "C104", courseName: "Operating Systems", credits: 4, department: "CSE",
16   instructorId: "I101" }
17 { _id: "C105", courseName: "Machine Learning", credits: 5, department: "IT",
18   instructorId: "I102" }
19
20 // instructors

```

```

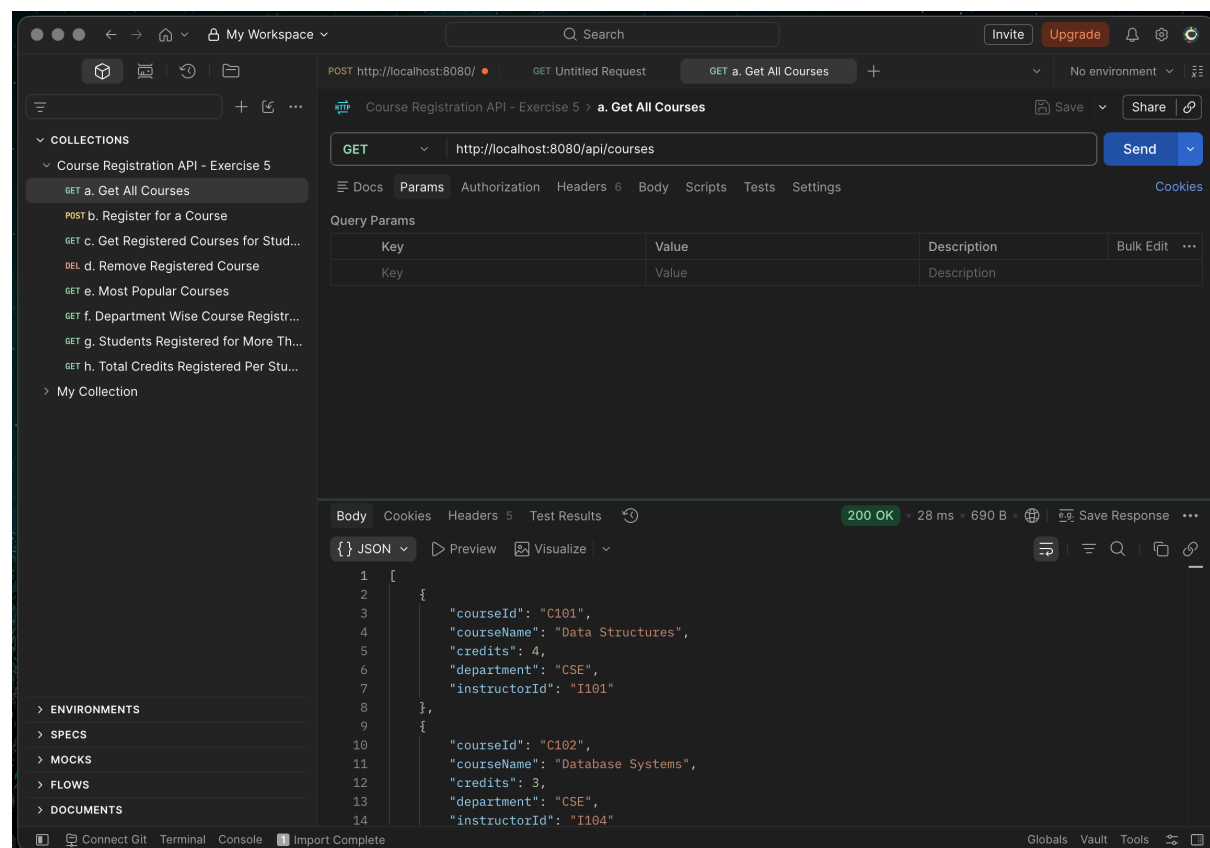
16 { _id: "I101", instructorName: "Dr. R. Kumar", experience: 12, department: "CSE"
    }
17 { _id: "I102", instructorName: "Dr. S. Meena", experience: 9, department: "IT"
    }
18 { _id: "I103", instructorName: "Dr. A. Raj", experience: 15, department: "ECE"
    }
19 { _id: "I104", instructorName: "Dr. P. Devi", experience: 7, department: "CSE"
    }
20
21 // registrations (created via POST /api/register, 11 total)
22 S101 -> C101, C102          S102 -> C101, C103, C105
23 S103 -> C102, C104          S104 -> C101, C103, C104
24 S105 -> C102

```

4. Postman API Testing and Output Screenshots

Each of the 8 REST endpoints specified in the exercise was imported into Postman as a collection (*Course Registration API – Exercise 5*) and executed against the running Spring Boot application (<http://localhost:8080>), which is connected to the local `demo/CourseDB` MongoDB database.

a. Retrieve All Courses – GET /api/courses



b. Register for a Course – POST /api/register

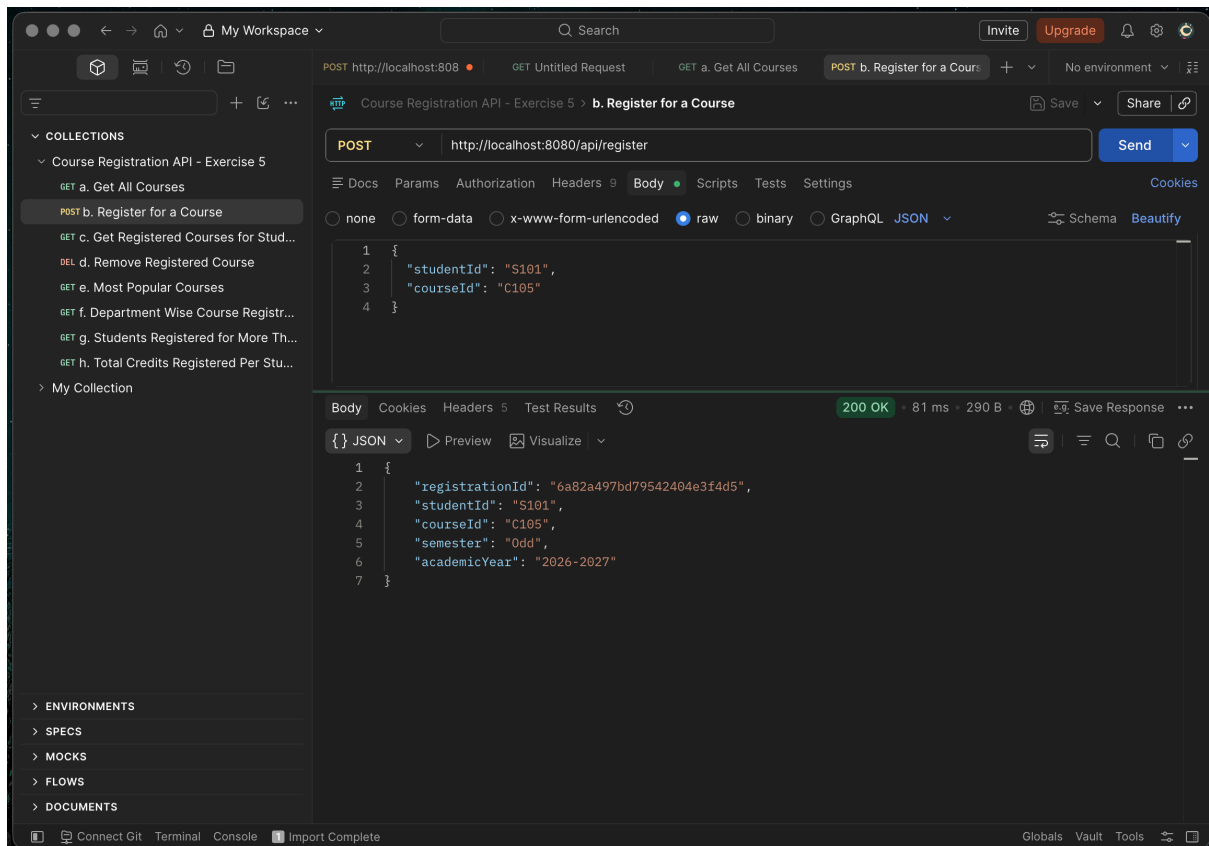
Request body:

```

1 {
2   "studentId": "S101",
3   "courseId": "C105"

```

4 }



The response echoes back the generated `registrationId` along with the auto-computed `semester` and `academicYear`.

c. Get Registered Courses for Student – GET /api/students/S101/courses

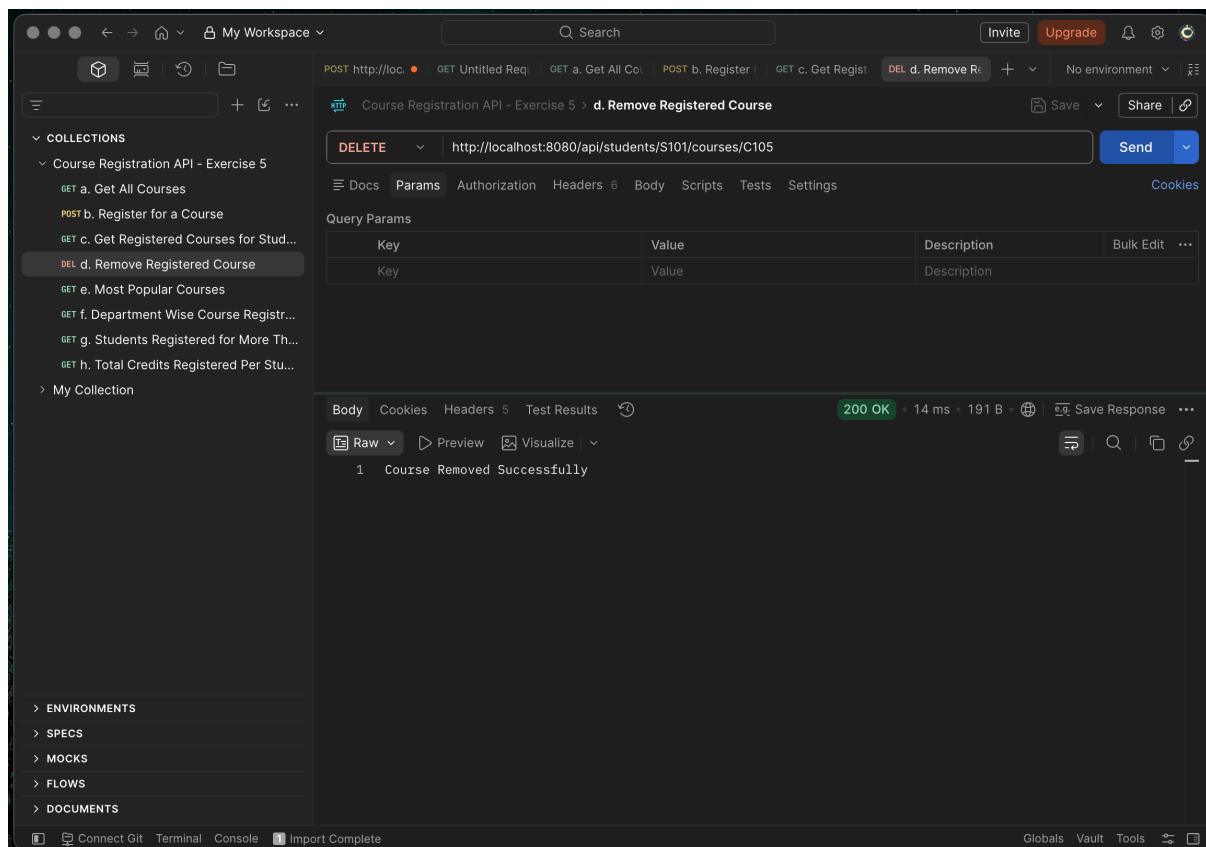
The screenshot shows a REST client interface with a collection named "Course Registration API - Exercise 5". The selected request is "GET c. Get Registered Courses for Student" with the URL "http://localhost:8080/api/students/S101/courses". The response is a 200 OK status with a response time of 28 ms and a body size of 546 B. The response body is a JSON array of three course registration objects for student S101.

Key	Value	Description	Bulk Edit
Key	Value	Description	

```
{
  "registrationId": "6a82a15d0105c8dcd11a53f9",
  "studentId": "S101",
  "courseId": "C101",
  "semester": "Odd",
  "academicYear": "2026-2027"
},
{
  "registrationId": "6a82a16f0105c8dcd11a53fa",
  "studentId": "S101",
  "courseId": "C102",
  "semester": "Odd",
  "academicYear": "2026-2027"
},
{
  "registrationId": "6a82a497bd79542404e3f4d5",
  "studentId": "S101",
  "courseId": "C105",
  "semester": "Odd",
  "academicYear": "2026-2027"
}
```

Student S101 now shows 3 registered courses (C101, C102, and the newly added C105).

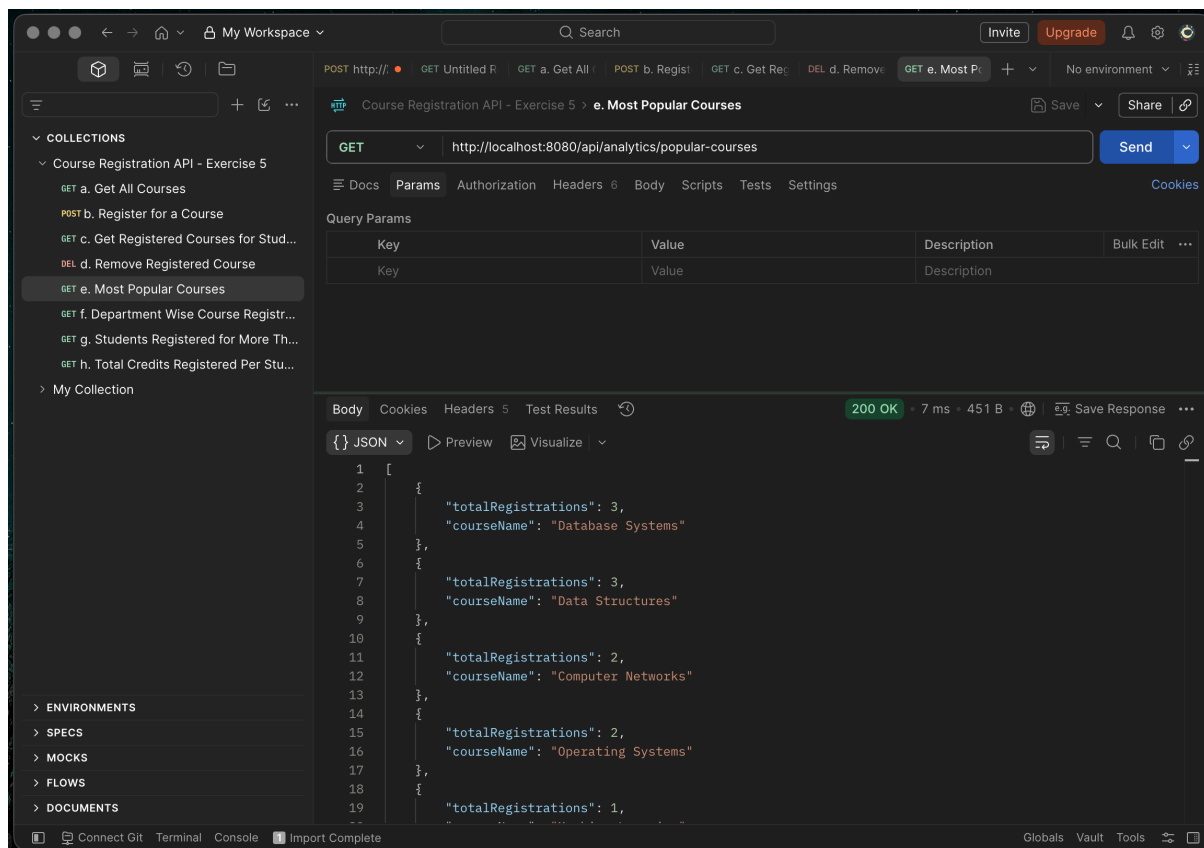
d. Remove Registered Course – DELETE /api/students/S101/courses/C105



The API returns the confirmation message *"Course Removed Successfully"* and the registration is deleted from MongoDB.

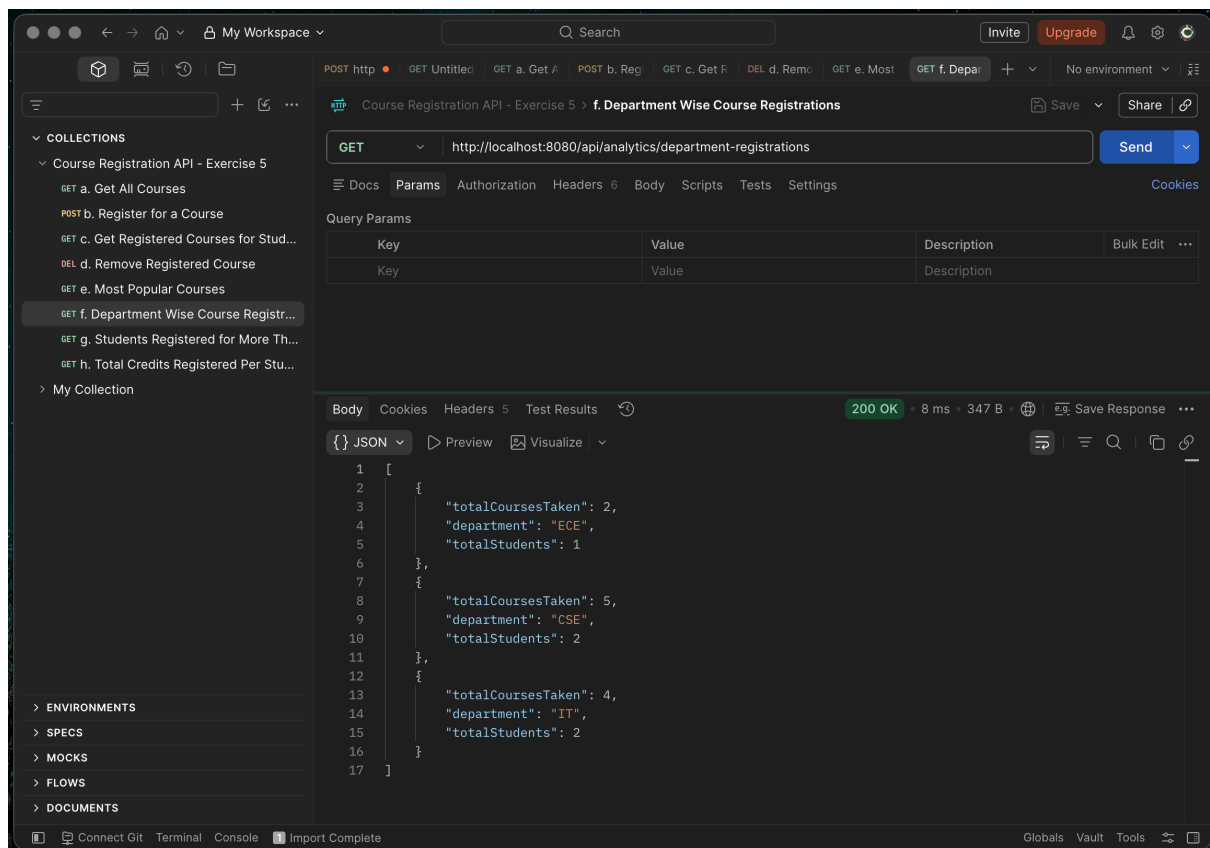
e. Most Popular Courses – GET /api/analytics/popular-courses

Aggregation pipeline: `$group` registrations by `courseId` to count total registrations, `$lookup` into `courses` to fetch the course name, then `$sort` descending by registration count.



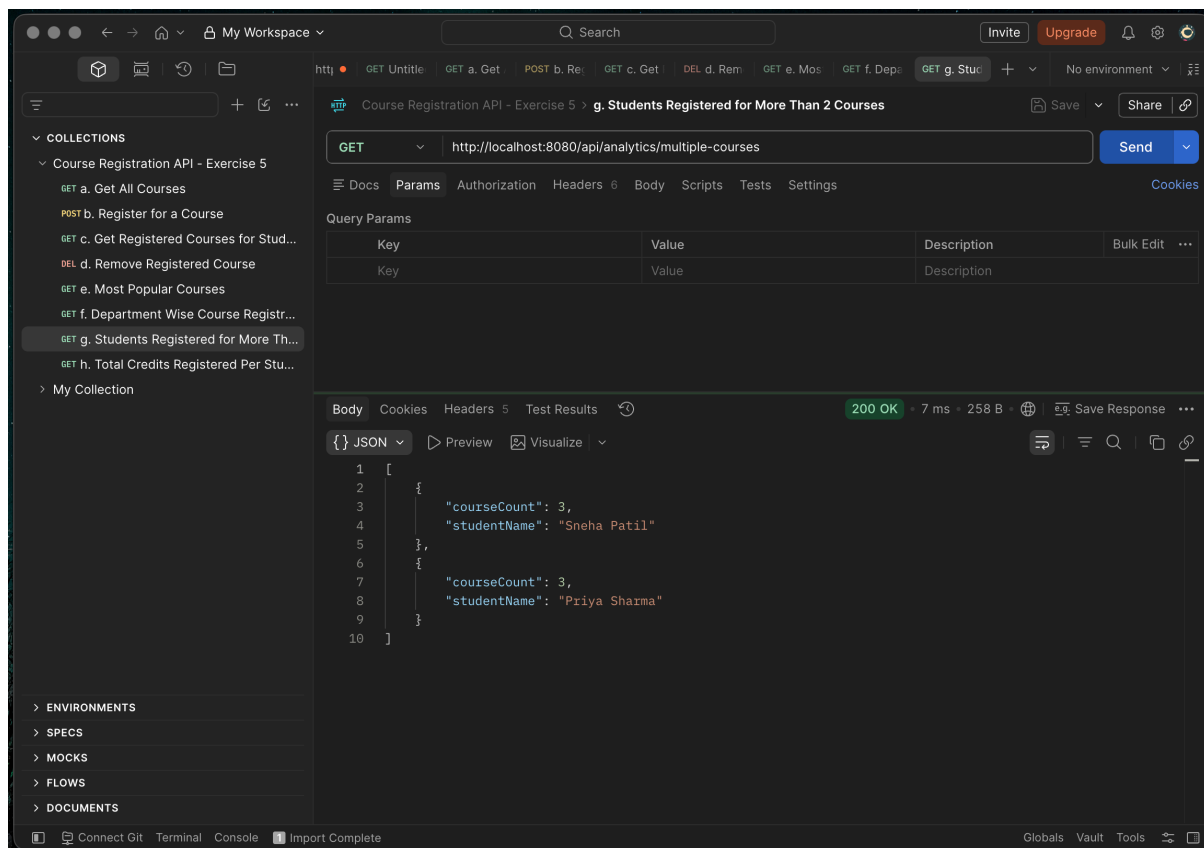
f. Department Wise Course Registrations – GET /api/analytics/department-registrations

Aggregation pipeline: \$lookup each registration's student to obtain their department, \$group by department while collecting distinct student IDs (\$addToSet) and counting total registrations, then \$project the distinct-student count as totalStudents.



g. Students Registered for More Than 2 Courses – GET /api/analytics/multiple-courses

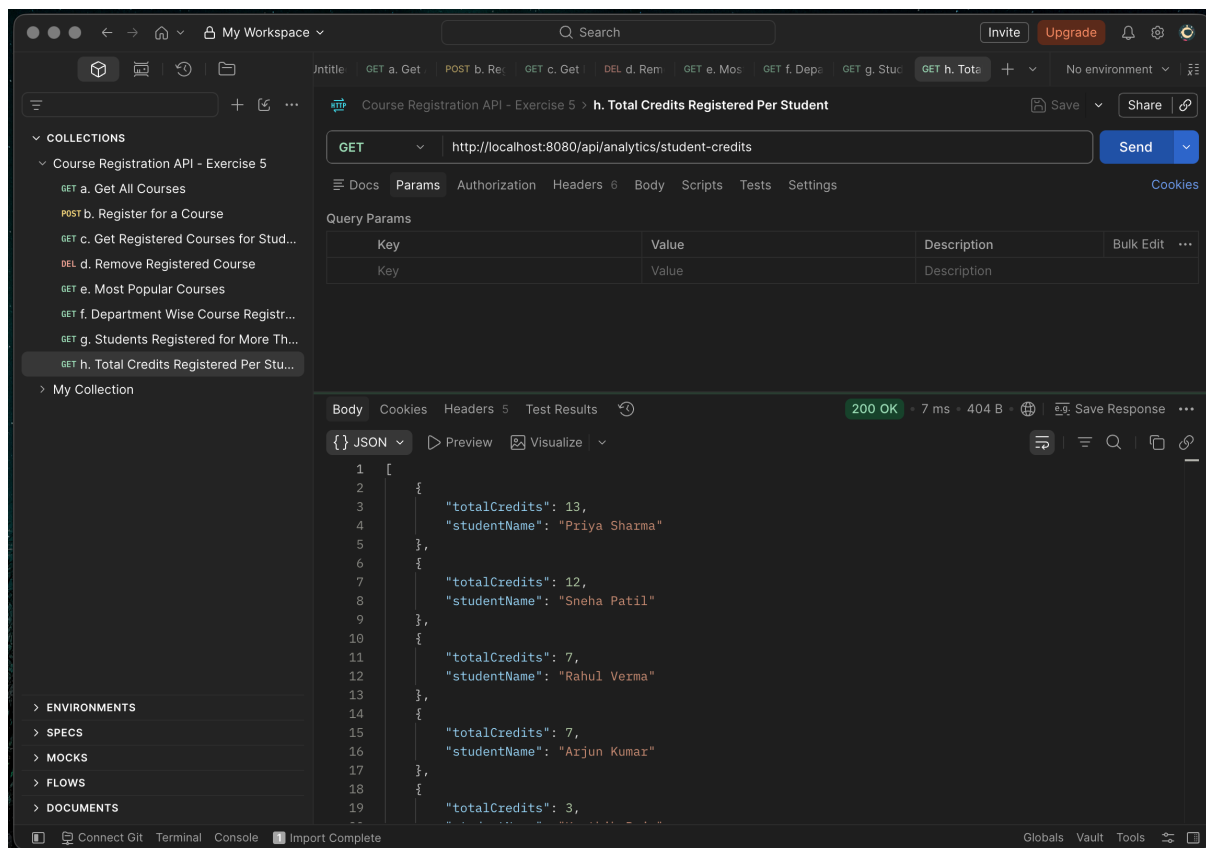
Aggregation pipeline: \$group registrations by studentId to get a course count per student, \$match where count > 2, \$lookup the student name, and \$sort descending.



Only *Priya Sharma* (*S102*) and *Sneha Patil* (*S104*) have registered for more than 2 courses.

h. Total Credits Registered Per Student – GET /api/analytics/student-credits

Aggregation pipeline: \$lookup each registration's course to obtain its credit value, \$group by studentId summing credits, \$lookup the student name, and \$sort descending by total credits.



5. Learning Outcomes

Through this lab exercise, I learned how to build a complete REST backend using Spring Boot integrated with MongoDB through Spring Data MongoDB. I understood how to model related collections (`students`, `courses`, `registrations`, `instructors`) using `@Document` and `@Id`, and how to expose CRUD operations through layered `Controller` → `Service` → `Repository` classes. I also gained hands-on experience writing multi-stage MongoDB aggregation pipelines (`$group`, `$lookup`, `$unwind`, `$match`, `$project`, `$sort`) with `MongoTemplate` to answer analytical questions that span multiple collections – something a single `find()` query cannot do. A key debugging lesson was that when a model's `@Id` field is mapped to MongoDB's `_id`, a `$lookup` against that collection must use `"_id"` as the `foreignField` rather than the Java field name, since no separate field is persisted for it. Finally, I learned to use Postman to build and run a full request collection against a locally running Spring Boot application, verifying both successful responses and validation error handling (HTTP 400 on blank `studentId/courseId`).